

Тема 3: Основни принципи за разработване на потребителски интерфейс, адаптиран за различни платформи – Desktop системи, Web базирани системи, Touch устройства (телефон, таблет). Избор на решение за реализация според заданието и изискванията на потребителя. Избор на документация.

Съдържание

1. Адаптивен потребителски интерфейс (Adaptive UI).....	3
2. Основни принципи за разработване на потребителски интерфейс, адаптиран за различни платформи – Desktop системи, Web базирани системи, Touch устройства (телефон, таблет).	3
? Простота (Simplicity).....	3
? Минимални действия (Minimal Actions).....	4
? Съгласуваност (Consistency)	4
? Обратна връзка (Feedback).....	4
? Яснота (Clarity).....	4
? Съобразен с платформата (Platform Adaptation)	5
3. Избор на решение за реализация според заданието и изискванията на потребителя	5
3.1. Видове изисквания:.....	6
А) Бизнес изисквания	6
Б) Потребителски изисквания.....	6
В) Изискванията към решението (Solution Requirements)	6
? Функционални изисквания	7
Видове функционални изисквания:	8
? Удостоверяване (Authentication).....	8
? Нива на оторизация (Authorization)	8
? Обработка на данни.....	8
? Потребителски интерфейс и потребителско изживяване (UI/UX).....	8
? Обработка на грешки и регистриране	8
? Архивиране и възстановяване.....	8
? Нефункционални изисквания	8
Видове нефункционални изисквания:	9
? Използваемост (Usability)	9
? Изискванията за сигурност (Security)	9
? Надеждност (Reliability).....	9
? Производителност (Performance)	9
3.2. Събиране и управление на изискванията	9
Събиране на изисквания	9
1. Идентифициране на заинтересованите страни	9
2. Събиране на информация	9

3. Документиране на изискванията	9
Управлението на изискванията	10
1. Приоритизиране.....	10
2. Управление на промените	10
3. Проследяване.....	10
4. Верификация и валидация	10
4. Избор на документация	10
Документацията за продукт.....	11
☐ Системна документация	11
☐ Потребителска документация	11
Документацията за процеса	12
Някои примери за документация :.....	12
Документът за изисквания към продукта (Product Requirement Document или PRD)	12
☐ бизнес правила	12
☐ потребителски истории (user stories)	12
☐ потребителски случаи (use case)	12
Документацията за дизайн на потребителския интерфейс (User Experience Design documentation или UX документация)	13
☐ Потребителски персонажи (Persona)	13
☐ Потребителски сценарий (User Scenario).....	13
☐ Карта на сценариите (Scenario Map)	13
☐ Карта на историята на потребителя (User Story Map)	13
☐ Стилово ръководство за UX (Style Guide)	13
☐ Карти на сайта (Site Map)	13
☐ Прототипи (Prototype)	13
☐ Макети (Mock-Ups)	13
☐ Схеми на потребителския поток или пътешествие на потребителя (User Flow and User Journey).....	13
☐ Доклади от тестове за употребимост (Usability Testing Reports).....	13
5. Software Development Lifecycle	14
- Планиране	14
- Анализ на изискванията	14
- Дизайн	14
- Разработка	14
- Тестване	14
- Интеграция и удостоверяване	14
- Внедряване и поддръжка	14

1. Адаптивен потребителски интерфейс (Adaptive UI)

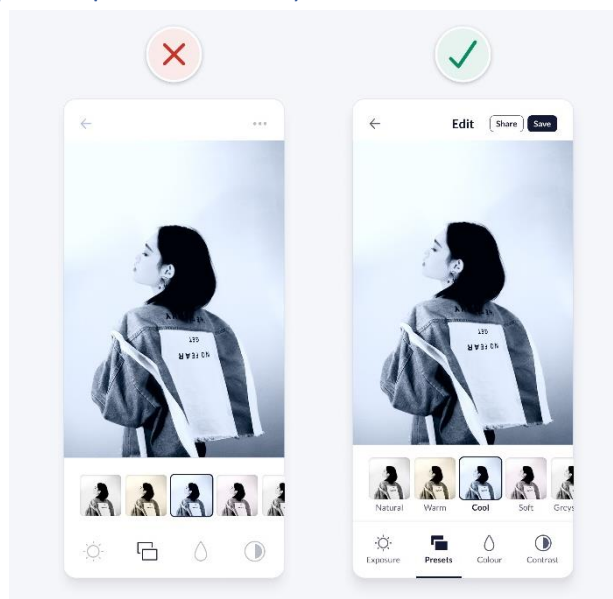
Адаптивният потребителски интерфейс (Adaptive UI) е дизайнерска практика, която се стреми да осигури оптимално изживяване на потребителите на различни устройства и екрани. Това включва създаването на уеб сайтове, приложения и други софтуерни интерфейси, които се адаптират и реагират на различни размери на екрана, разделителни способности, ориентации и други характеристики на устройството, на което се използва. Този подход на дизайн е особено важен в света на мобилните устройства, където разнообразието на екрани е много голямо. Адаптивният UI помага на потребителите да имат еднакво удобство и функционалност, независимо от това дали използват смартфон, таблет, лаптоп или друго устройство.



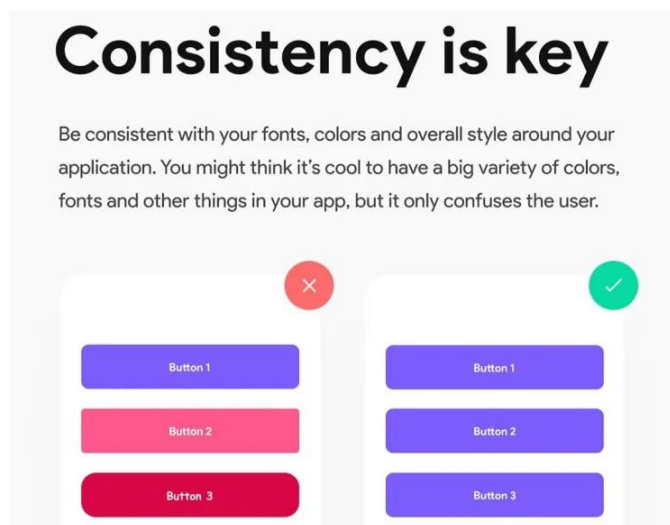
2. Основни принципи за разработване на потребителски интерфейс, адаптиран за различни платформи – Desktop системи, Web базирани системи, Touch устройства (телефон, таблет).

Разработването на потребителски интерфейс за различни платформи изисква придържане към специфичните характеристики. Някои от основните принципи са:

- **Простота (Simplicity)** - Интерфейсът трябва да бъде възможно най-прост и интуитивен за използване. Сложните функционалности трябва да бъдат представени по начин, който не създава объркване от страна на потребителите.



- **Минимални действия (Minimal Actions)** - Потребителите трябва да могат да постигнат целите си с минимален брой действия. Менютата и навигационните елементи трябва да бъдат оптимизирани, за да намалят излишното преминаване между различни екрани и функции.

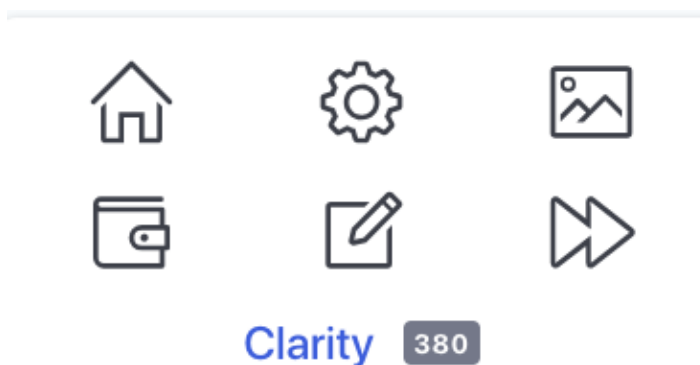


- **Съгласуваност (Consistency)** - Всички части на интерфейса трябва да са съгласувани помежду си, както и с общата стилистика на платформата. Това включва единна цветова палитра, шрифтове, стил на бутоните и други елементи.

- **Обратна връзка (Feedback)** - Интерфейсът трябва да предоставя незабавна обратна връзка на потребителите за техните действия. Например, когато потребител натисне бутон, трябва да има визуален сигнал за потвърждение, че действието е прието.



- **Яснота (Clarity)** - Всички елементи на интерфейса трябва да бъдат ясни и разбираеми за потребителите. Това включва ясни икони, текстове и инструкции, които да улеснят потребителите в тяхната навигация и използване на приложението или уебсайта.



- **Съобразен с платформата (Platform Adaptation)**

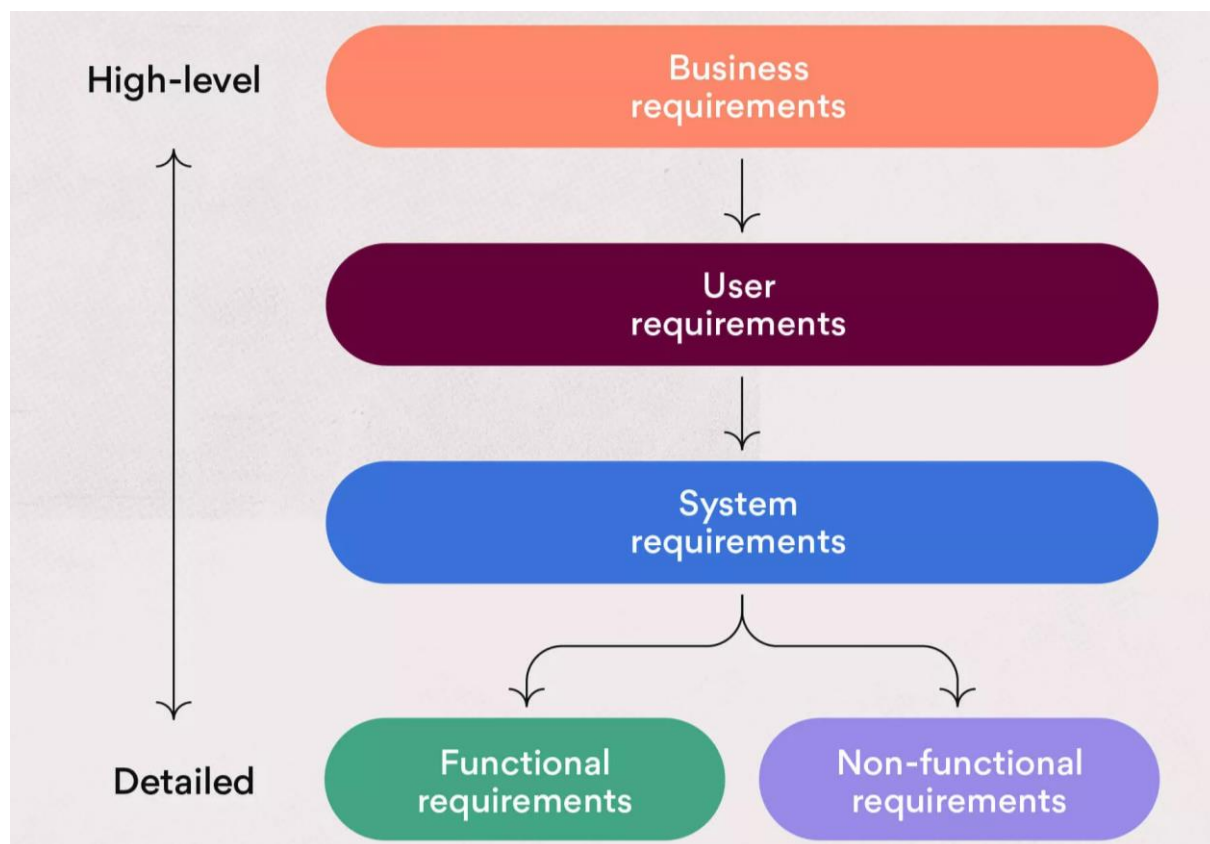
- Всяка платформа има свои специфични функции и възможности. Интерфейсът трябва да бъде съобразен с тези характеристики. Например, интерфейсът за десктоп компютър може да включва повече функционалност и сложност, докато интерфейсът за мобилни устройства трябва да бъде по-опростен и лесен за използване.



Тези принципи представляват основата за създаването на потребителски интерфейс, който е адаптиран за различни платформи, като десктоп, уеб и тъч устройства. Разработчиците трябва да се стремят да интегрират тези принципи във всеки етап от процеса на проектиране и разработка на софтуерни приложения.

3. Избор на решение за реализация според заданието и изискванията на потребителя

Ясно дефинираните изисквания са основни знаци по пътя към успешен проект. Те установяват официално споразумение между клиента и доставчика, че и двамата работят за постигане на една и съща цел.

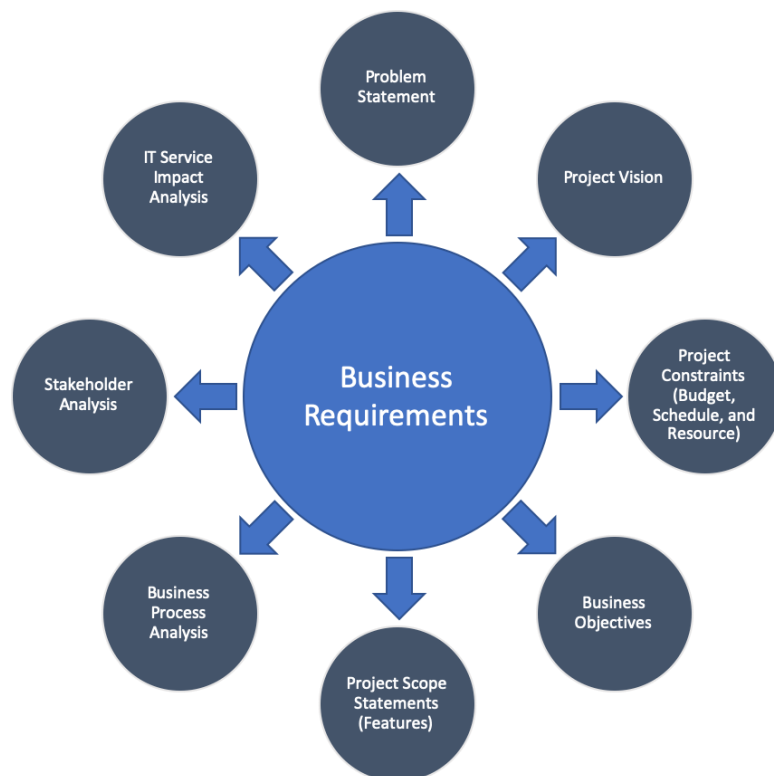


3.1. Видове изисквания:

А) Бизнес изисквания

Тези изисквания включват общи заявления за цели, обективи и нужди. Бизнес изискванията нямат детайли или конкретни характеристики. Те просто посочват проблема и бизнес целта, която трябва да бъде постигната, като например:

- увеличение на приходите/пропускоспособността/достъпността на клиентите,
- намаляване на разходите/грешките,
- подобро обслужване на клиентите и други.



Б) Потребителски изисквания

Тази група изисквания отразява нуждите на конкретни групи заинтересовани страни (мениджъри, персонал, клиенти и т.н.) и определя какво очакват те от определено решение. Те служат като връзка между общите бизнес изисквания и конкретните изисквания за решение. Те могат да включват, например, възможността за създаване на различни отчети, преглед на историята и статуса на поръчките, управление на базите данни на клиентите и други.



В) Изискванията към решението (Solution Requirements)

Описват специфични характеристики, които продуктът трябва да притежава, за да отговори на нуждите на заинтересованите страни и на самия бизнес. Те попадат в две големи групи.

Функционалните изисквания определят какво трябва да прави даден продукт и какви са неговите характеристики и функции.

Нефункционалните изисквания описват общите свойства на системата. Те са известни също като качествени атрибути.

Функционалните и нефункционалните изисквания са две основни категории изисквания в разработката на софтуер. Всеки тип играе жизненоважна роля при определянето на характеристиките и работата на решението.



❖ *Функционални изисквания*

Функционалните изисквания са продуктови характеристики или функции, които разработчиците трябва да внедрят, за да позволят на потребителите да изпълнят своите задачи. Затова е от съществено значение те да бъдат ясни както за екипа за разработка, така и за заинтересованите страни. Като цяло функционалните изисквания описват поведението на системата при определени условия.

FUNCTIONAL REQUIREMENTS

WHAT THE SYSTEM SHOULD DO

Функционалните изисквания варират за различните видове софтуер. Примерни изисквания за уебсайт:

- Системата изпраща имейл за потвърждение, когато бъде създаден нов потребителски акаунт.
- Функция за търсене позволява на потребителите да търсят съдържание/артикули чрез въвеждане на заявката в лентата за търсене.
- Потребителят може да прегледа артикулите в количката, да промени количеството им или да ги премахне преди плащане и т.н.



PRODUCT FEATURES



USER REQUIREMENTS

Това са някои общи функционални изисквания. По-специализираните софтуерни системи ще имат по-специфични изисквания.

Видове функционални изисквания:

- **Удостоверяване (Authentication)** - Тази група е за потвърждаване на самоличността на потребител, преди да разреши достъп до системата, включително въвеждане на потребителски имена и пароли, биометрична проверка или многофакторно удостоверяване.
- **Нива на оторизация (Authorization)** - Тези изисквания имат за цел да определят и контролират нивата на достъп на различни потребители в системата. Например, администратор може да има пълен достъп до системата, докато обикновен потребител има ограничен достъп до определени функции.
- **Обработка на данни** - Тези изисквания могат да включват въвеждане на данни, валидиране, съхранение и извличане.
- **Потребителски интерфейс и потребителско изживяване (UI/UX)** - Това са изискванията, свързани с дизайна и взаимодействието на елементите на системата. Тяхната цел е да гарантират, че интерфейсът е лесен за използване и отговаря на нуждите на потребителите.
- **Обработка на грешки и регистриране** - Тези изисквания определят как системата трябва да обработва грешките и да ги регистрира, например дефиниране на съобщения за грешки, стъпки за отстраняване на неизправности и поддържане на регистрационни файлове за системни дейности.
- **Архивиране и възстановяване** - Това са изискванията за архивиране на данни и процеси за възстановяване на системата, гарантиращи целостта на данните и наличността на системата в случай на повреда.

Това са само някои от видовете функционални изисквания.

❖ Нефункционални изисквания

Нефункционалните изисквания не са свързани с функционалността на системата, а по-скоро определят как системата трябва да работи. Те са от решаващо значение за осигуряване на използваемостта, надеждността и ефективността на системата, като често оказват влияние върху цялостното потребителско изживяване.

Някои примери за нефункционални изисквания са:

- Системата трябва да може да се справи с 20 милиона потребители без влошаване на производителността.

- Програма, работеща на Windows 10, трябва да може да работи на Windows 11 без никаква промяна в нейното поведение и производителност.

Видове нефункционални изисквания:

- **Използваемост (Usability)** - определя колко трудно ще бъде за потребителя да научи и работи със системата.
- **Изискванията за сигурност (Security)** - гарантират, че има защита от неоторизиран достъп до системата и нейните съхранени данни. Той взема предвид различни нива на оторизация и удостоверяване за различните потребителски роли.
- **Надеждност (Reliability)** - определя колко вероятно е софтуерът да работи без повреда за дадено време. Надеждността намалява поради грешки в кода, хардуерни повреди или проблеми с други системни компоненти.
- **Производителност (Performance)** - атрибут на качеството, който описва отзивчивостта на системата към различни потребителски взаимодействия. Лошата производителност води до отрицателно потребителско изживяване. Освен това застрашава безопасността на системата, когато е претоварена.

NON-FUNCTIONAL REQUIREMENTS

HOW THE SYSTEM SHOULD DO IT



PRODUCT PROPERTIES



USER EXPECTATIONS

Това са само някои от функционалните изисквания.

3.2. Събиране и управление на изискванията

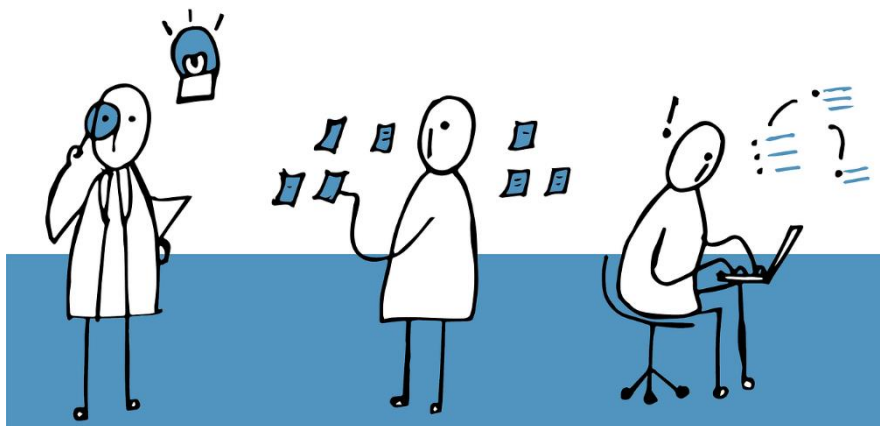
Събирането и управлението на изискванията са две от съществените фази във всеки проект.

Събиране на изисквания - включва дефинирането на специфичните нужди и очаквания на заинтересованите страни за нова система, софтуерно приложение или какъвто и да е проект. По същество, става въпрос за разбиране на това, което трябва да бъде направено.

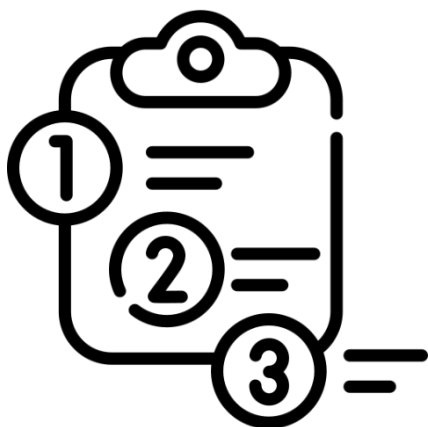
В кратки думи, процесът на събиране на изисквания има три основни части:

1. **Идентифициране на заинтересованите страни** - определяне на кой ще бъде засегнат от проекта и кой трябва да даде своя принос.
2. **Събиране на информация** - използване на интервюта, анкети и други методи за събиране на информация за нуждите и очакванията на заинтересованите страни.
3. **Документиране на изискванията** - записване на събраната информация във вид на документация.

Обикновено бизнес анализаторите са отговорни за събирането на изискванията, въпреки че понякога продуктите собственици или мениджърите на продукти също участват.



Управлението на изискванията – използва се за гарантиране, че всички изисквания се спазват по време на процеса на разработка и че всички промени се управляват и документиранат внимателно. Основните компоненти на управлението на изискванията са:



1. Приоритизиране - определяне кои изисквания са най-съществени и трябва да бъдат реализирани първи.

2. Управление на промените - справяне с промените в изискванията по контролиран начин, за да се предотврати увеличаването на обхвата.

3. Проследяване - следене на всички изисквания и уверяване, че те се адресират през целия жизнен цикъл на проекта.

4. Верификация и валидация - осигуряване, че разработената система отговаря на всички идентифицирани изисквания и че тези изисквания изпълняват своята предназначена цел.

Следващият голям въпрос, който трябва да се проучи, е кои документи и формати можете да използвате за улавяне на изискванията.

4. Избор на документация

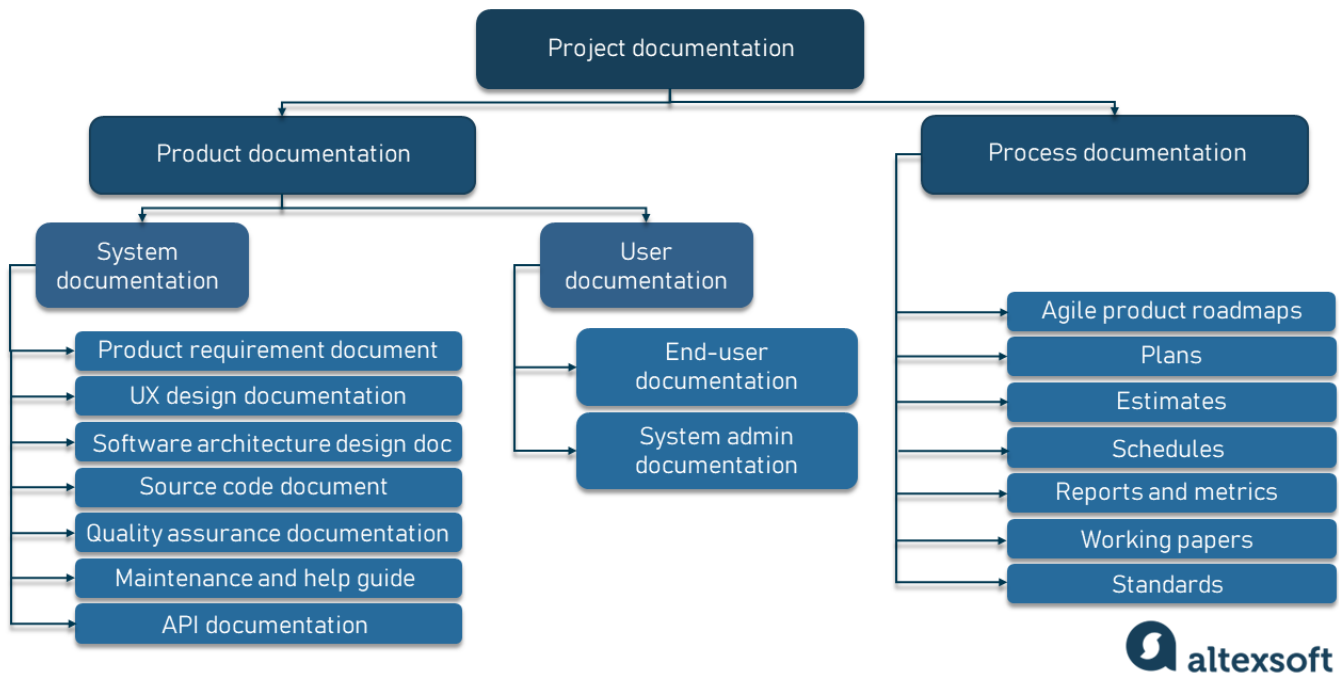
Документацията при проектиране на потребителски интерфейси представлява набор от материали, които описват и документирант различните аспекти на интерфейсия дизайн и функционалност. Тази документация е от съществено значение за разработчиците, дизайнерите и всички участници в процеса на разработка на софтуерни приложения или уеб сайтове.

Всички документации за софтуер може да бъдат разделени на две основни категории:

- Документация за продукт
Проектиране на потребителски интерфейси

- Документация за процеса

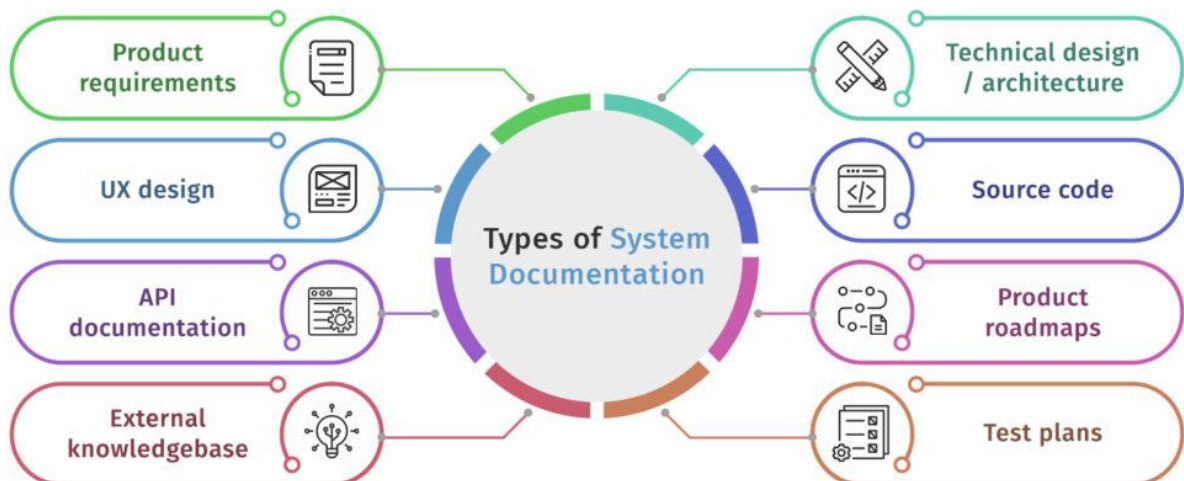
SOFTWARE DOCUMENTATION TYPES



Документацията за продукт - описва продукта, който се разработва, и предоставя инструкции за изпълнение на различни задачи с него. В общи линии, документацията за продукт включва изисквания, технически спецификации, бизнес логика и ръководства.

Има два основни типа документация за продукт:

- Системна документация** - представлява документи, които описват самата система и нейните части. Това включва документи за изисквания, решения за дизайн, описания на архитектурата и програмен код.



- Потребителска документация** - покрива ръководства, които се подготвят главно за крайните потребители на продукта и системните администратори. Потребителската документация включва уроци, потребителски ръководства, ръководства за отстраняване на проблеми, инсталационни и справочни ръководства.

Проектиране на потребителски интерфейси

Документацията за процеса - представлява всички документи, произведени по време на разработката и поддръжката, които описват целия процеса. Общи примери за документи, свързани с процеса, са стандарти и документация за проекта, като планове за проекти, графики на тестове, доклади, записки от срещи или дори делова кореспонденция.



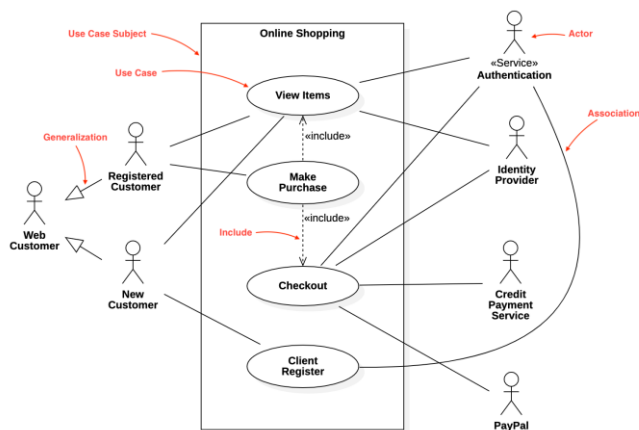
Основната разлика между документацията за процеса и тази за продукта е, че първата записва процеса на разработка, а втората описва продукта, който се разработва.

Някои примери за документация :

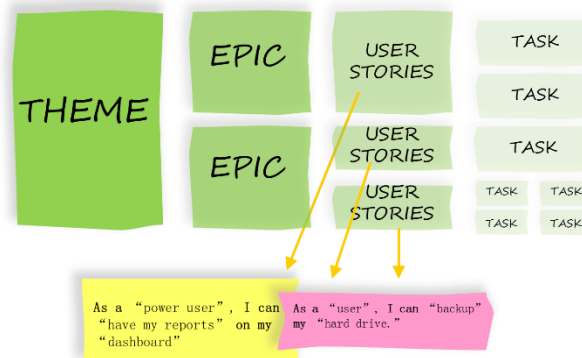
Документът за изисквания към продукта (Product Requirement Document или PRD) предоставя информация за функционалността на системата. Обикновено изискванията са изявления за това, какво трябва да прави една система. Те могат да бъдат функционални и нефункционални. Така, че документът за изисквания към продукта съдържа:

- *бизнес правила*
- *потребителски истории (user stories)*
- *потребителски случаи (use case)*

и т.н., и трябва да бъде ясен и не трябва да бъде обширен и солиден текст. Трябва да съдържа информация, за да се определи целта на продукта, функциите, възможностите, поддръжката и поведението му.



USER STORIES



Документацията за дизайн на потребителския интерфейс (User Experience Design documentation или UX документация) започва от етапа на изискванията и продължава през всички етапи на разработката, включително тестовите и етапите след пускането на продукта. Процесът на UX дизайн включва изследвания, прототипиране, тестване на употребимостта. Етапът на изследване включва:

- *Потребителски персонажи (Persona)*
- *Потребителски сценарий (User Scenario)*
- *Карта на сценариите (Scenario Map)*
- *Карта на историята на потребителя (User Story Map)*
- *Стилово ръководство за UX (Style Guide)*



USER RESEARCH

По време на етапа на прототипиране и дизайн, дизайнерът на потребителския интерфейс често работи с изходни материали и актуализира документацията на равнище с другите членове на екипа, включително собственика на продукта, дизайнерите на потребителски интерфейси и инженерите. Най-често срещаните документи, произвеждани на тези етапи, са:

- *Карти на сайта (Site Map)*
- *Прототипи (Prototype)*
- *Макети (Mock-Ups)*
- *Схеми на потребителския поток или пътешествие на потребителя (User Flow and User Journey)*
- *Доклади от тестове за употребимост (Usability Testing Reports)*



PROTOTYPE

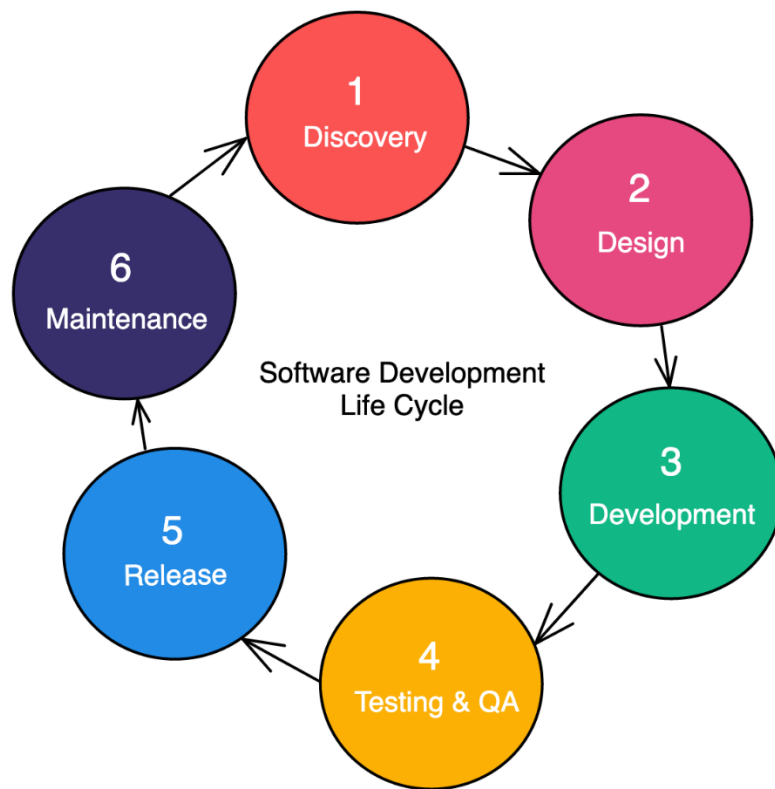
Има още множество видове документация, които могат да бъдат разгледани на [изображението](https://www.altexsoft.com/blog/technical-documentation-in-software-development-types-best-practices-and-tools/) по-горе. За повече информация може да посетите този линк: <https://www.altexsoft.com/blog/technical-documentation-in-software-development-types-best-practices-and-tools/>

Всички тези видове документация са от съществено значение за успешното изпълнение на проекта и осигуряват яснота, съгласуваност и ефективност в работата на екипа. Те помагат на всички участници в проекта да разберат целите, изискванията и процесите и да работят съгласувано за постигането на общата цел.

5. Software Development Lifecycle

Жизненият цикъл за разработката на софтуер (Software Development Lifecycle - SDLC) е процес, който описва етапите и активностите, които се извършват при създаването на софтуерен продукт. Въпреки че конкретните етапи и методологии могат да варират в зависимост от проекта и организацията, общо взето, този цикъл включва следните основни фази:

- **Планиране** - На този етап се определят целите на проекта, анализират се изискванията на клиента и се планират ресурсите и времето за изпълнение.
- **Анализ на изискванията** - В тази фаза се анализират, документират и проверяват изискванията на клиента за софтуерния продукт.
- **Дизайн** - Тук се проектира архитектурата на софтуерния продукт, включително структура, интерфейси и технологии.
- **Разработка** - Програмистите започват да пишат код, базиран на дизайна и спецификациите, дефинирани в предходните етапи.
- **Тестване** - На този етап се извършва систематично тестване на софтуерния продукт, за да се осигури, че той отговаря на изискванията и работи правилно.
- **Интеграция и удостоверяване** - В тази фаза се интегрират различните компоненти на софтуерния продукт и се удостоверява, че те работят заедно по правилен начин.
- **Внедряване и поддръжка** - След като софтуерният продукт е тестван и одобрен, той се внедрява в продукцията и се предоставя на клиентите. Също така се осигурява поддръжка и подпомагане за продукта през неговия жизнен цикъл.



Тези етапи могат да бъдат изпълнени последователно (като waterfall модел) или итеративно (като agile методологии). Освен това, някои организации могат да добавят допълнителни етапи или да използват комбинации от различни методи, в зависимост от спецификата на проекта.